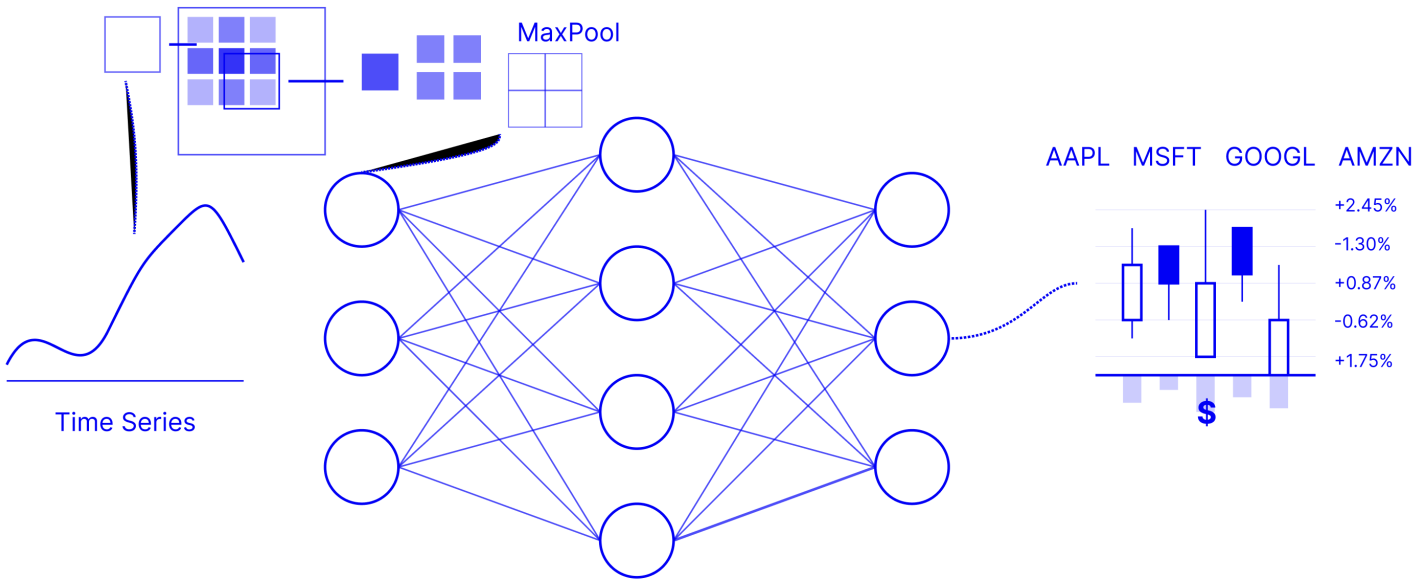
Deep Stock Representation Learning

based on: 2018 ICASSP: 10.1109/ICASSP.2018.8462215

Guosheng Hu, Yuxin Hu, Kai Yang *et al.*
Project by: Amitesh Pandey



$y = \sigma(Wx + b)$ $\nabla L = \partial L / \partial w$ $Conv(I, K) = \sum\sum I(m,n)K(i-m,j-n)$

OUTLINE

MOTIVATION

Mean-Variance Theory (Markowitz 1952): For n assets, let w_i $(1 \leq i \leq n)$ be the constituent weighting of asset i in the optimal portfolio, such that $\sum_i w_i = 1$. Then, the return of this portfolio R_p is naturally

$$\mathbb{E}[R_p] = \sum_{i=1}^{n} w_i \mathbb{E}[R_i]$$

where $\mathbb{E}[R_i]$ is the expected return of asset i.

Mean-Variance Theory (Markowitz 1952): For n assets, let w_i $(1 \leq i \leq n)$ be the constituent weighting of asset i in the optimal portfolio, such that $\sum_i w_i = 1$. Then, the return of this portfolio R_p is naturally

$$\mathbb{E}[R_p] = \sum_{i=1}^{n} w_i \mathbb{E}[R_i]$$

where $\mathbb{E}[R_i]$ is the expected return of asset i.

The variance ($\approx$ risk) of the portfolio is

$$\sigma_p^2 = \sum_{i=1}^{n} w_i^2 \sigma_i^2 = \sum_{i=1}^{n} \sum_{j \neq i} w_i w_j \sigma_i \sigma_j \rho_{ij}$$

We use the covariance matrix Σ, populated with ρ_{ij}. The estimates for Σ are very error-prone. Additionally, the estimation of Σ for n risky assets takes n days of data for any reasonable accuracy (Rothschild and Chamberlain 1982).

Mean-Variance Theory (Markowitz 1952): For n assets, let w_i $(1 \leq i \leq n)$ be the constituent weighting of asset i in the optimal portfolio, such that $\sum_i w_i = 1$. Then, the return of this portfolio R_p is naturally

$$\mathbb{E}[R_p] = \sum_{i=1}^{n} w_i \mathbb{E}[R_i]$$

where $\mathbb{E}[R_i]$ is the expected return of asset i.

The variance ($\approx$ risk) of the portfolio is

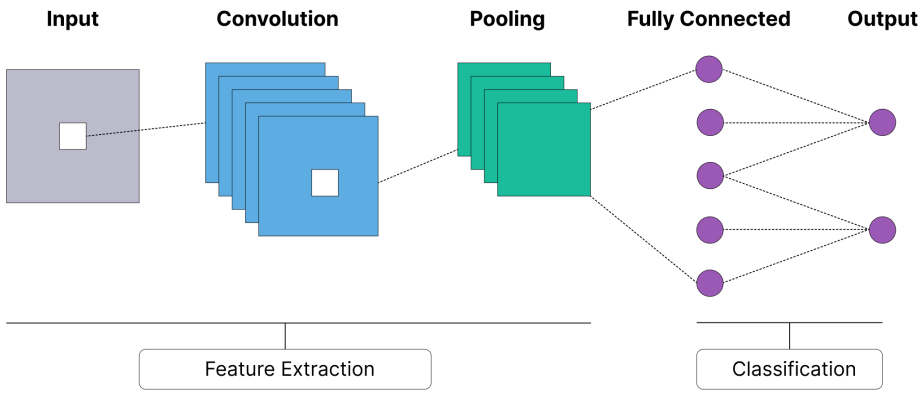
$$\sigma_p^2 = \sum_{i=1}^{n} w_i^2 \sigma_i^2 = \sum_{i=1}^{n} \sum_{j \neq i} w_i w_j \sigma_i \sigma_j \rho_{ij}$$

We use the covariance matrix Σ, populated with ρ_{ij}. The estimates for Σ are very error-prone. Additionally, the estimation of Σ for n risky assets takes n days of data for any reasonable accuracy (Rothschild and Chamberlain 1982).

Typically, Σ is very computationally expensive, $\approx O(n^2 d)$ (if we are considering d features, assuming $d < n$). Finally, Σ is also typically populated with some linear measure of correlation, like the correlation coefficient, or the Pearson ratio. This fails to track the non-linear dynamics that markets tend to follow.

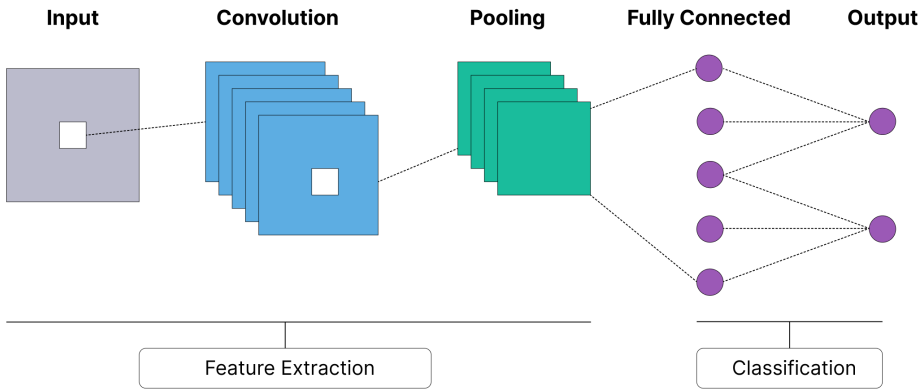
MOTIVATION

Convolutional Neural Networks: Type of deep learning network designed to process and analyze grid-like data, such as images, by using convolutional layers to detect patterns.

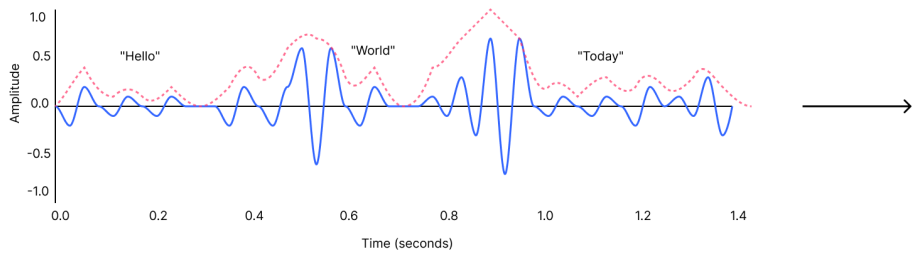


MOTIVATION

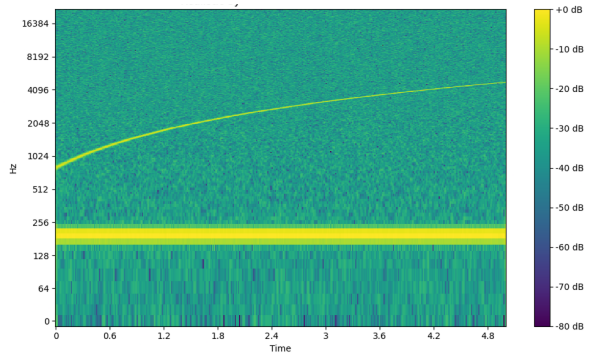
Convolutional Neural Networks: Type of deep learning network designed to process and analyze grid-like data, such as images, by using convolutional layers to detect patterns.



EX: Speech Recognition: 1D audio waveform → spectrogram



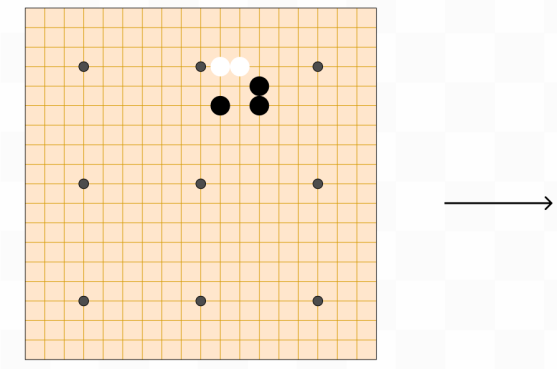
Audio Waveform



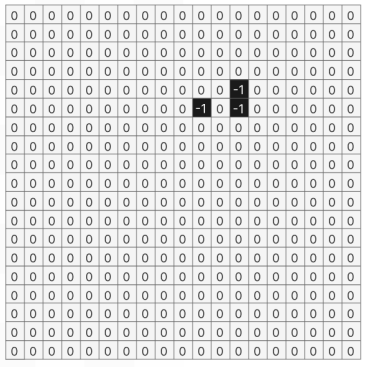
Spectrogram

MOTIVATION

EX: Board $\rightarrow$ gray scale image (Alpha Go)



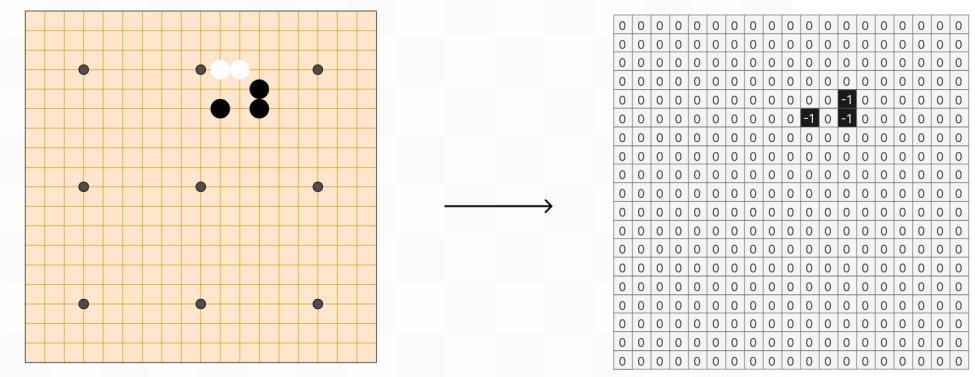
19 by 19 Go Board



Gray Scale Image Encoding

MOTIVATION

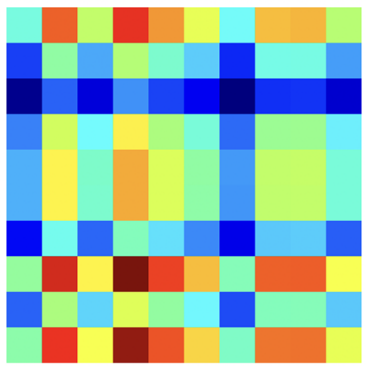
EX: Board $\rightarrow$ gray scale image (Alpha Go)



19 by 19 Go Board **Gray Scale Image Encoding**

EX: Visually identifying rank

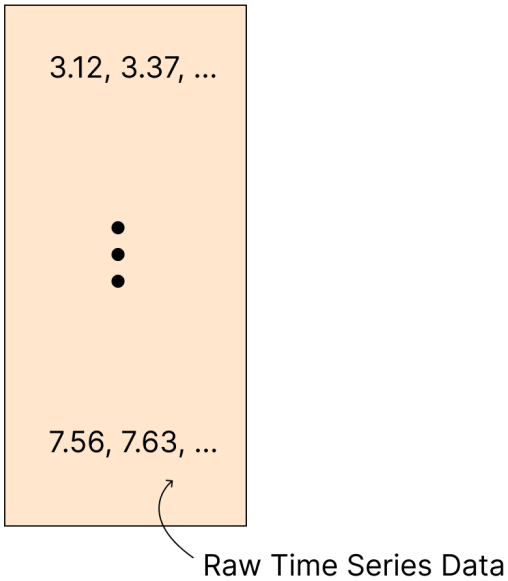
$$\begin{bmatrix} 12 & 5 & 7 & 3 & 9 & 1 & 4 & 8 & 2 & 6 \\ 3 & 14 & 6 & 2 & 7 & 8 & 1 & 5 & 9 & 10 \\ 9 & 4 & 13 & 6 & 3 & 7 & 2 & 1 & 8 & 5 \\ 6 & 1 & 8 & 15 & 4 & 2 & 9 & 7 & 3 & 10 \\ 7 & 2 & 5 & 9 & 11 & 6 & 3 & 1 & 4 & 8 \\ 1 & 6 & 3 & 4 & 8 & 12 & 7 & 2 & 9 & 5 \\ 5 & 7 & 2 & 1 & 6 & 3 & 14 & 8 & 10 & 4 \\ 8 & 9 & 4 & 7 & 1 & 5 & 6 & 13 & 2 & 3 \\ 2 & 3 & 1 & 8 & 5 & 4 & 10 & 6 & 12 & 7 \\ 4 & 8 & 9 & 5 & 2 & 10 & 3 & 11 & 1 & 6 \end{bmatrix}$$



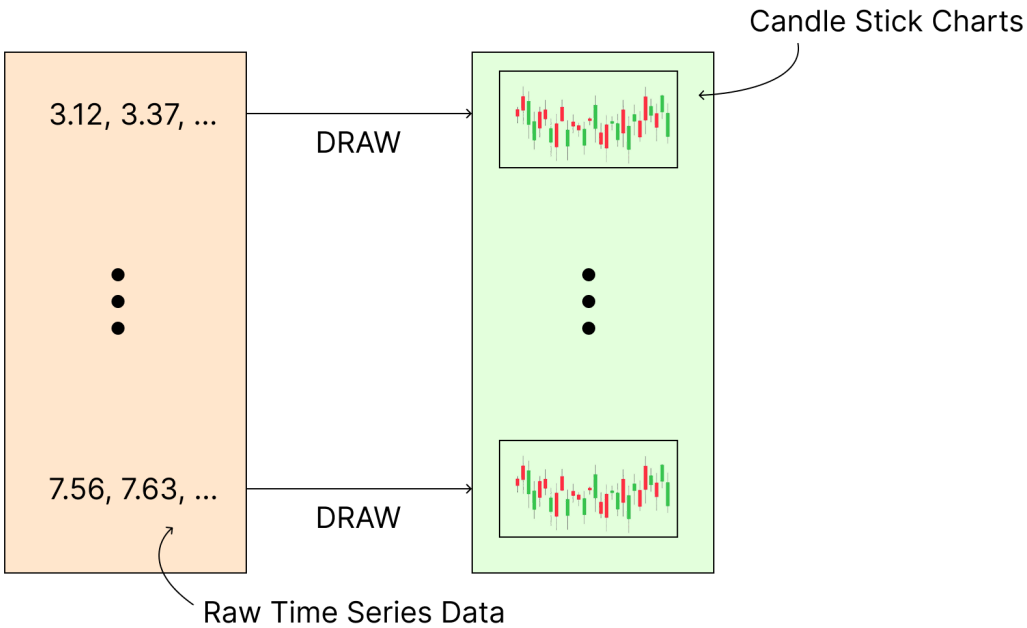
Matrix **Density**

METHODOLOGY

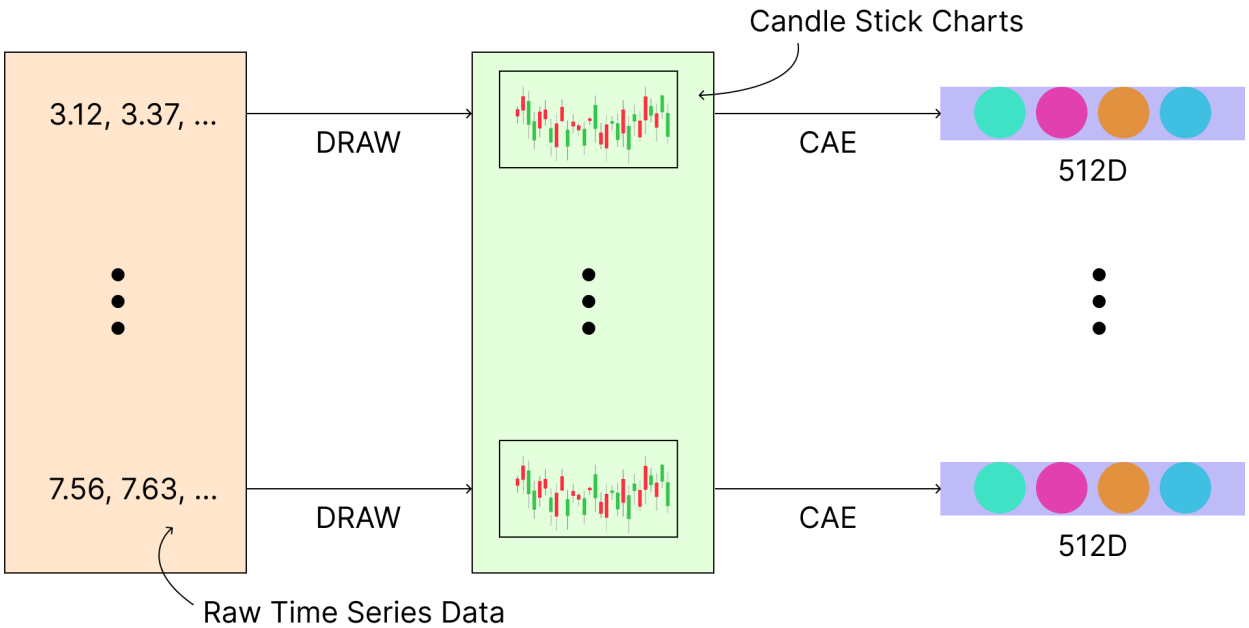
Pipeline:



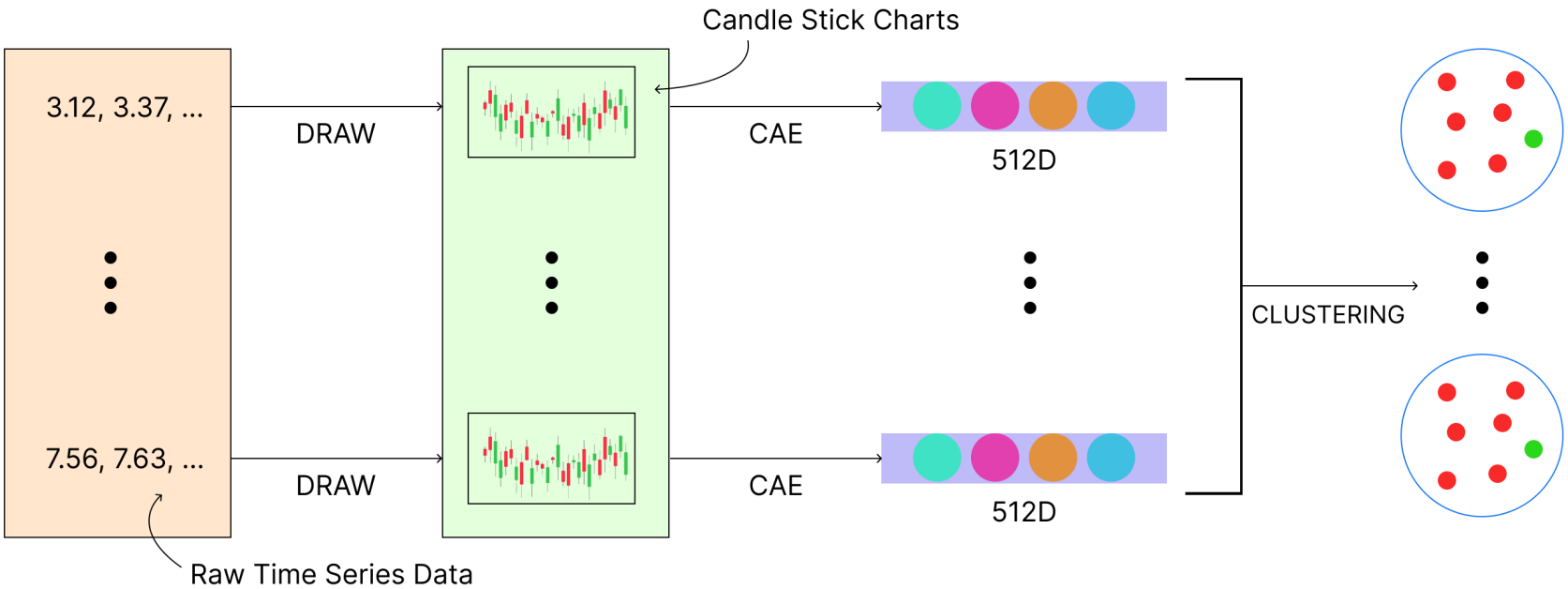
Pipeline:


Candle Stick Charts
3.12, 3.37, ...
DRAW
7.56, 7.63, ...
DRAW
Raw Time Series Data

Pipeline:

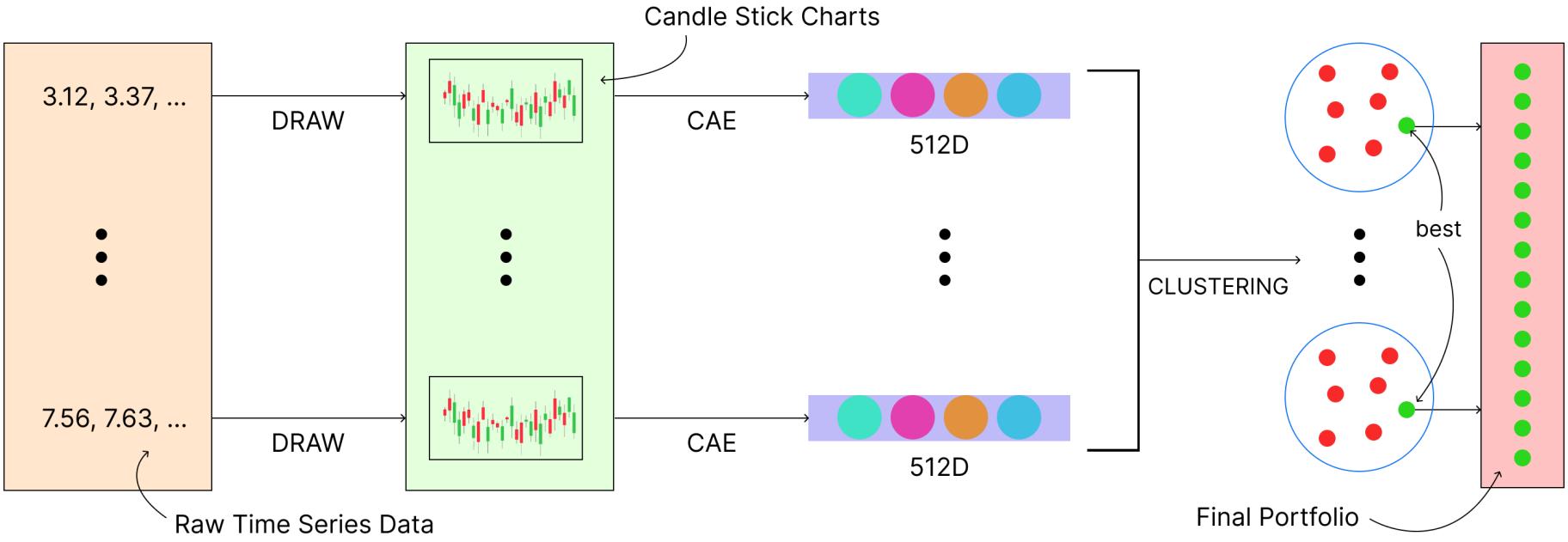

Candle Stick Charts
3.12, 3.37, ...
DRAW
CAE
512D
7.56, 7.63, ...
DRAW
CAE
512D
Raw Time Series Data

Pipeline:


Candle Stick Charts
3.12, 3.37, ...
DRAW
CAE
512D
CLUSTERING
7.56, 7.63, ...
DRAW
CAE
512D
Raw Time Series Data

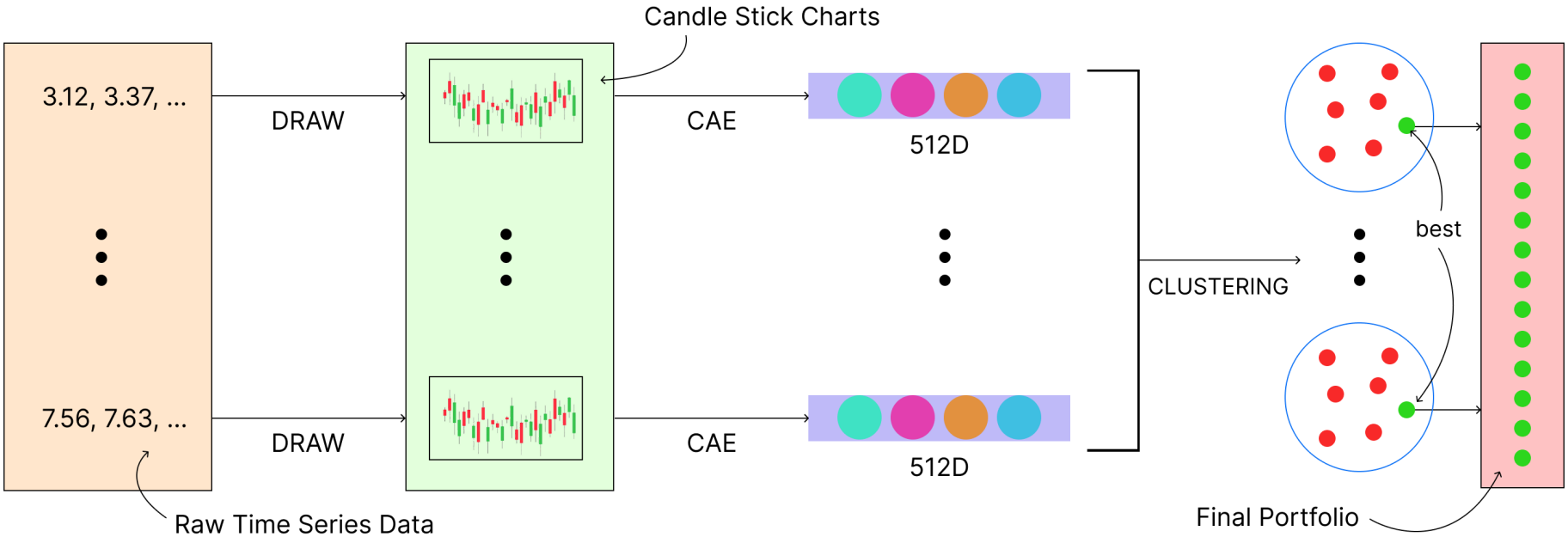
METHODOLOGY

Pipeline:



Claims:

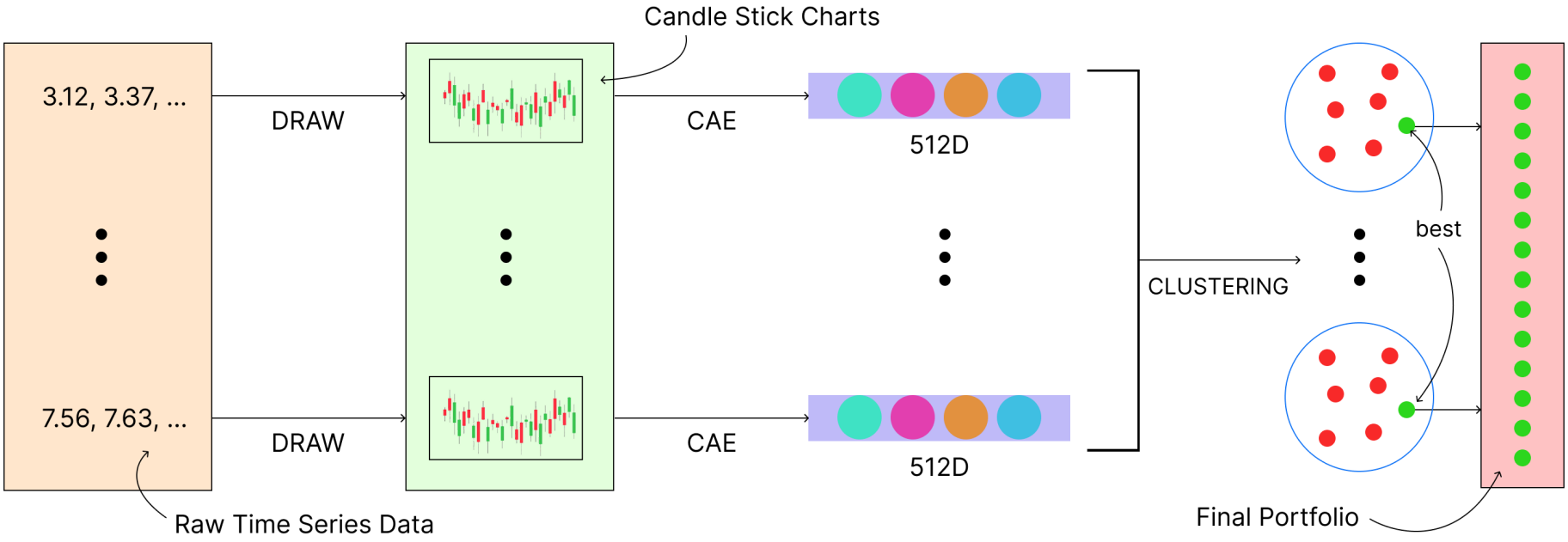
Pipeline:



Claims:

1. Accounts for translational invariance

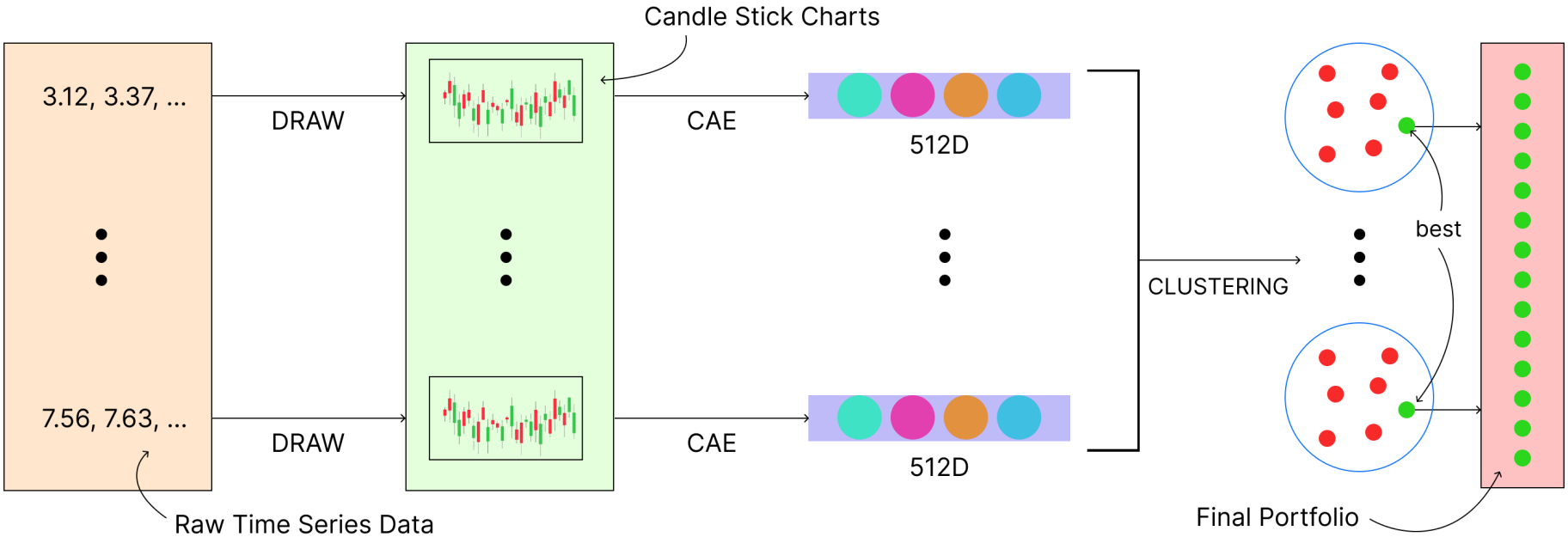
Pipeline:



Claims:

1. Accounts for translational invariance
2. Accounts for the non-linear dynamics of markets

Pipeline:



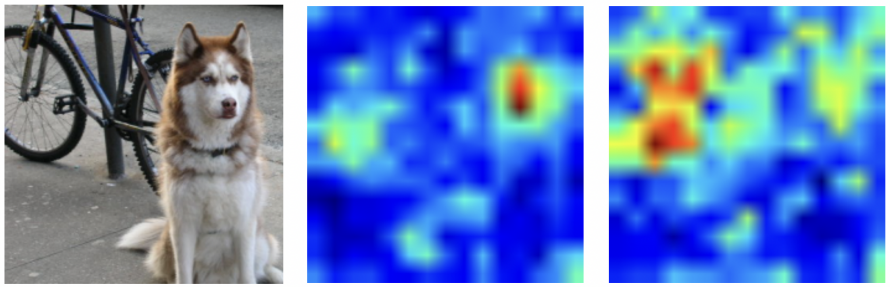
Claims:

1. Accounts for translational invariance
2. Accounts for the non-linear dynamics of markets
3. Not much more expensive than traditional techniques

Specifications:

Specifications:

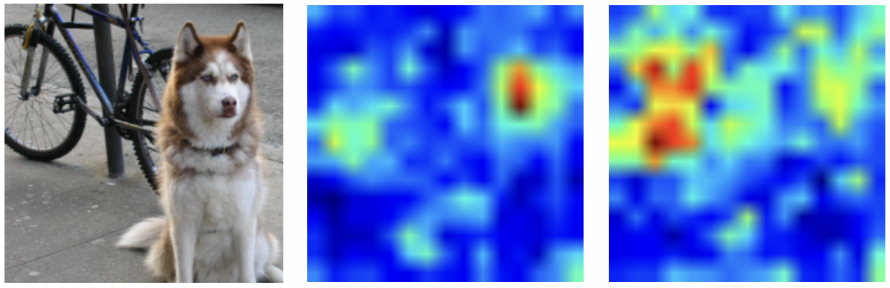
- Convolutional Autoencoder (CAE): For learning deep features



(a) Original Images (b) Animal Trained (c) Vehicle Trained

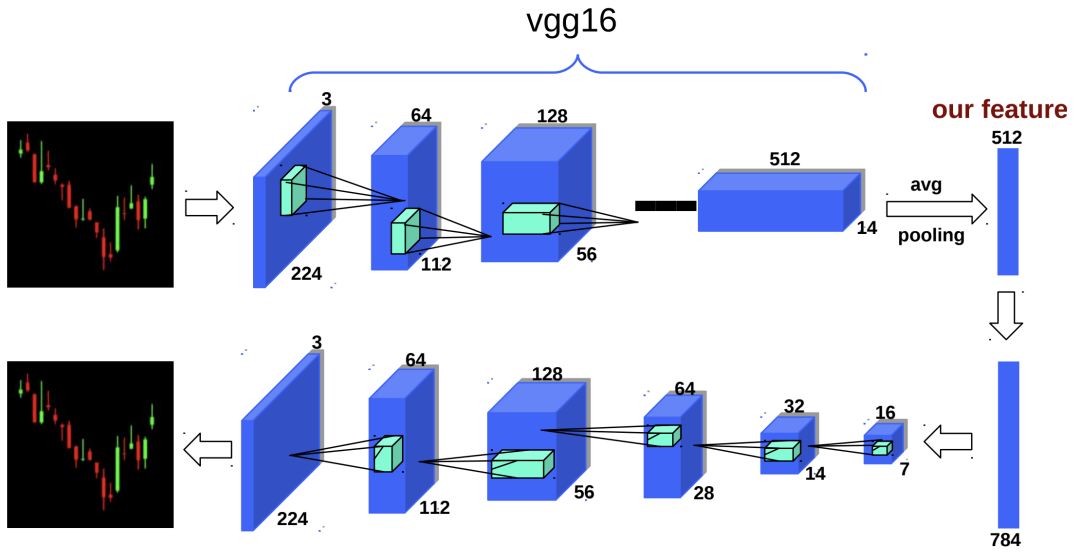
Specifications:

- Convolutional Autoencoder (CAE): For learning deep features



(a) Original Images (b) Animal Trained (c) Vehicle Trained

- VGG16 based CAE, outputs 512D encoded data that preserves as much information as possible. Achieved after removing the final 4096D FC layer.



Modularity-Based Clustering:

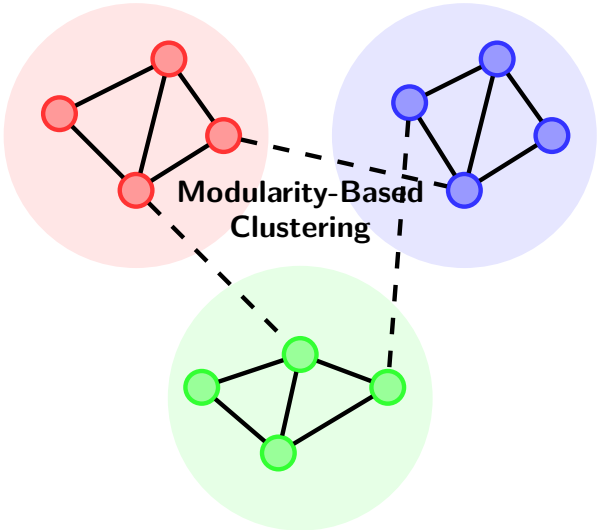
- Optimizes network structure via modularity Q:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Modularity-Based Clustering:

- Optimizes network structure via modularity Q:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

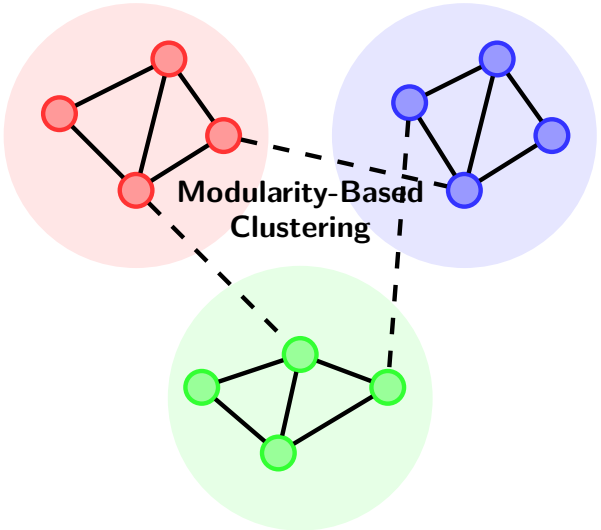


Modularity-Based Clustering:

- Optimizes network structure via modularity Q:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

- **Advantages:**
  - Automatically determines number of clusters
  - Excels with network/graph data
  - Captures complex relationships
  - Detects communities of varying sizes

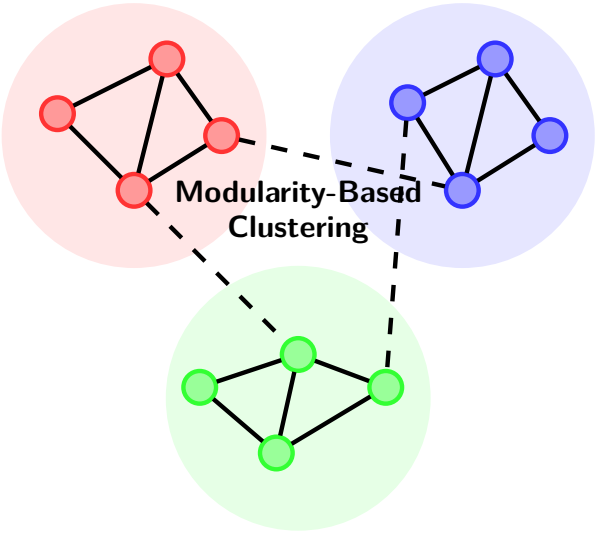


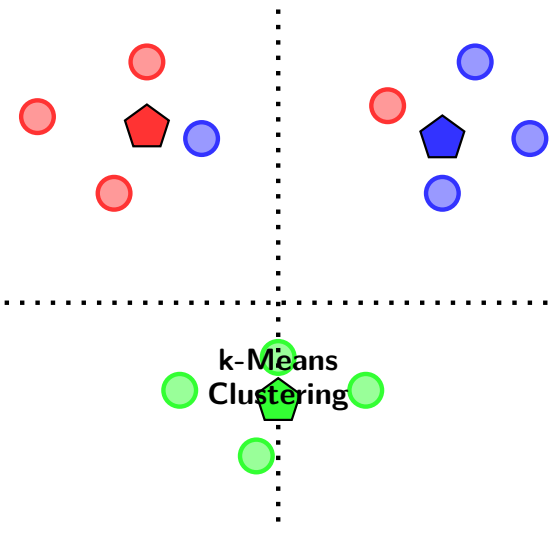
Modularity-Based Clustering:

- Optimizes network structure via modularity Q:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

- **Advantages:**
  - Automatically determines number of clusters
  - Excels with network/graph data
  - Captures complex relationships
  - Detects communities of varying sizes





Comparison	**Modularity**	**k-Means**	**Winner**
Requires known k	No	Yes	Modularity
Graph/network data	Excellent	Poor	Modularity
Euclidean space	Limited	Excellent	k-Means
Computational complexity	$O(n^2 \log n)$	O(nkdi)	Context-dependent
Irregular clusters	Handles well	Struggles	Modularity

Table: Difference between *k*-means and modularity clustering

Comparison	Modularity	k-Means	Winner
Requires known k	No	Yes	Modularity
Graph/network data	Excellent	Poor	Modularity
Euclidean space	Limited	Excellent	k-Means
Computational complexity	$O(n^2 \log n)$	$O(nkdi)$	Context-dependent
Irregular clusters	Handles well	Struggles	Modularity

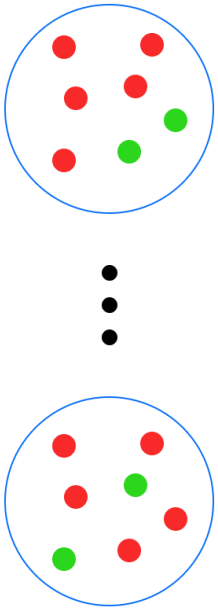
Table: Difference between *k*-means and modularity clustering

Portfolio Construction

Comparison	Modularity	k-Means	Winner
Requires known k	No	Yes	Modularity
Graph/network data	Excellent	Poor	Modularity
Euclidean space	Limited	Excellent	k-Means
Computational complexity	$O(n^2 \log n)$	$O(nkdi)$	Context-dependent
Irregular clusters	Handles well	Struggles	Modularity

Table: Difference between *k*-means and modularity clustering

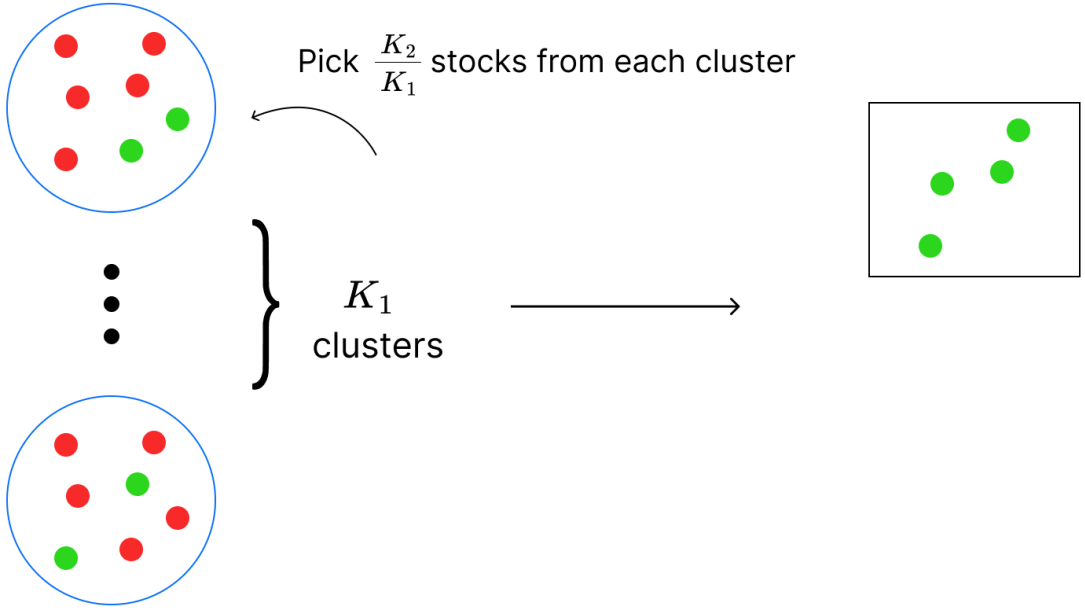
Portfolio Construction



Comparison	Modularity	k-Means	Winner
Requires known k	No	Yes	Modularity
Graph/network data	Excellent	Poor	Modularity
Euclidean space	Limited	Excellent	k-Means
Computational complexity	$O(n^2 \log n)$	$O(nkdi)$	Context-dependent
Irregular clusters	Handles well	Struggles	Modularity

Table: Difference between *k*-means and modularity clustering

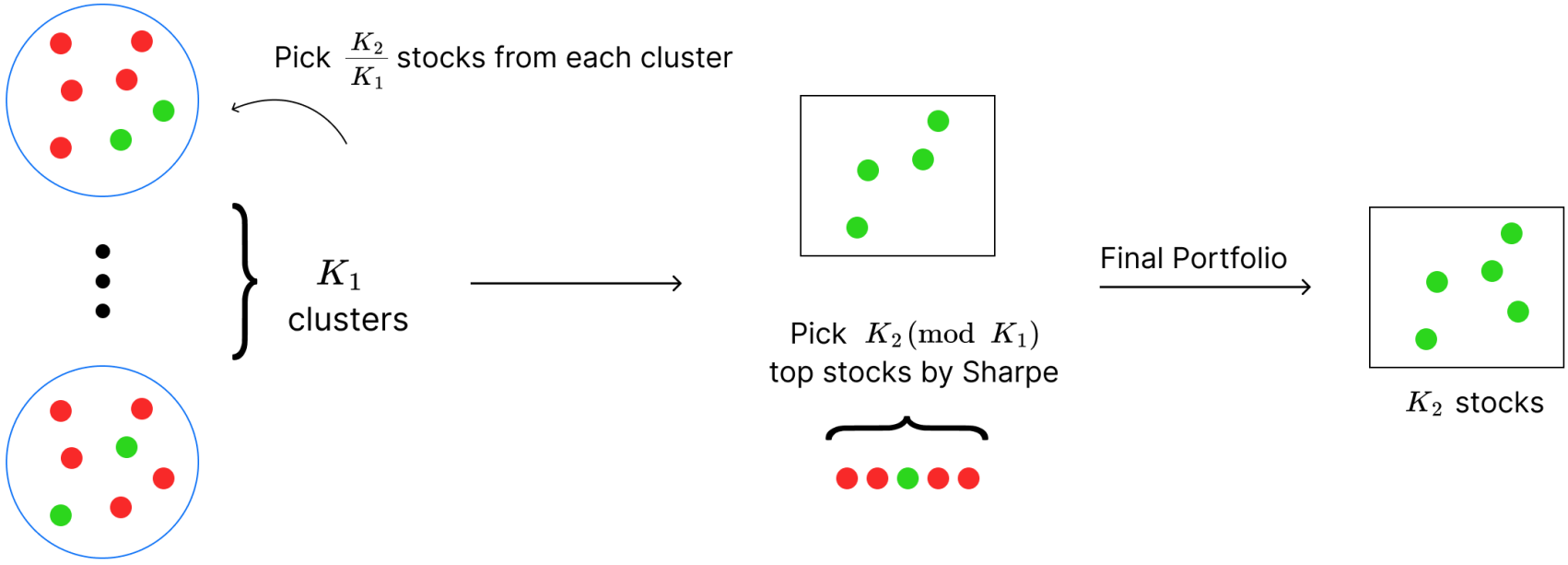
Portfolio Construction



Comparison	Modularity	k-Means	Winner
Requires known k	No	Yes	Modularity
Graph/network data	Excellent	Poor	Modularity
Euclidean space	Limited	Excellent	k-Means
Computational complexity	$O(n^2 \log n)$	$O(nkdi)$	Context-dependent
Irregular clusters	Handles well	Struggles	Modularity

Table: Difference between *k*-means and modularity clustering

Portfolio Construction



RESULTS

Market Settings

Assets Considered: Financial Times 100 (FTSE 100) of the London Stock Exchange
Period Considered: 4th Jan 2000 to 14th May 2017

Market Settings

Assets Considered: Financial Times 100 (FTSE 100) of the London Stock Exchange
Period Considered: 4th Jan 2000 to 14th May 2017

Model Settings

```
Images: 224 by 224 pixel Candlestick Charts
Learning Batch Size: 64
Learning Rate: 0.001
Learning Rate Decay Rate: 0.1
```

Market Settings

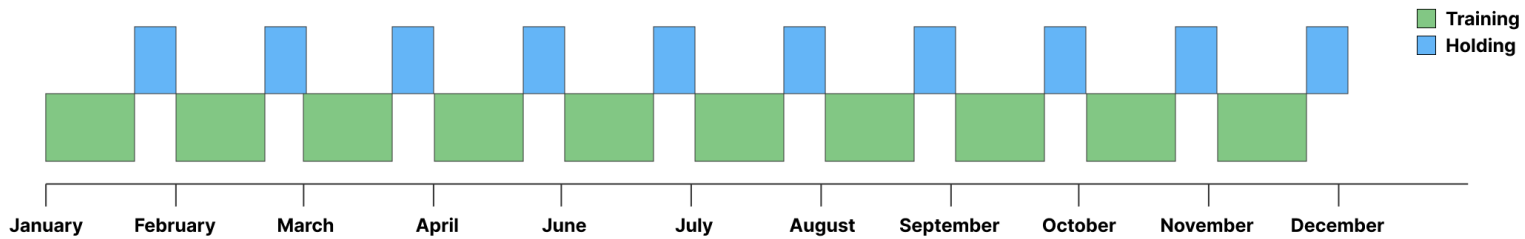
Assets Considered: Financial Times 100 (FTSE 100) of the London Stock Exchange
Period Considered: 4th Jan 2000 to 14th May 2017

Model Settings

```
Images: 224 by 224 pixel Candlestick Charts
Learning Batch Size: 64
Learning Rate: 0.001
Learning Rate Decay Rate: 0.1
```

Investment Testing

For every 20 trading days, hold the selected portfolio for 10 days. Repeat the process for the entire duration. Analogous to training and testing in ML.



Plot of the *t*-SNE Results:

This is an interesting approach to test the legitimacy of the deep feature.

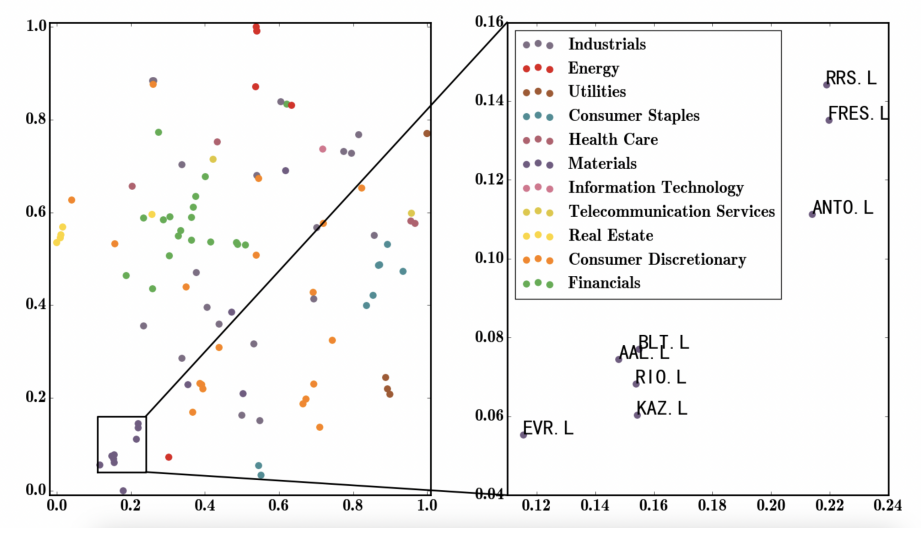


Figure: *t*-SNE plot of concatenated 512D CAE vectors for 2012

Plot of the *t*-SNE Results:

This is an interesting approach to test the legitimacy of the deep feature.

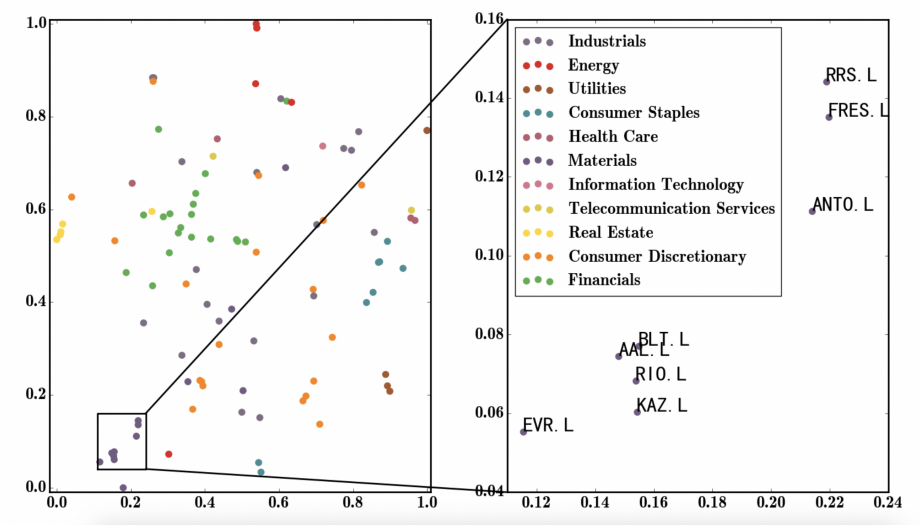


Figure: *t*-SNE plot of concatenated 512D CAE vectors for 2012

> *We can see the stocks with similar semantics (industrial sector) are represented close to each other in the learned feature space. This illustrates the efficacy of our CAE and learned feature for capturing semantic information about stocks.*

Trends in the portfolio's worth against the FTSE100 across horizons

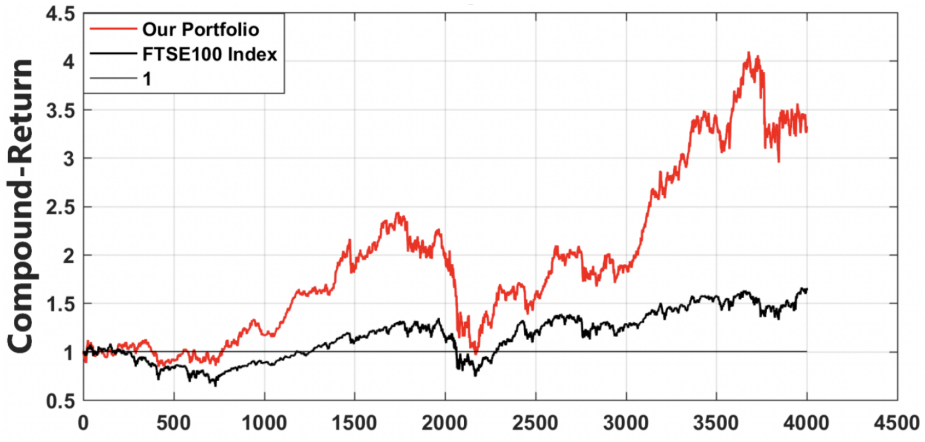


Figure: Long-term (Jan 2000 to Jun 2016)

Trends in the portfolio's worth against the FTSE100 across horizons

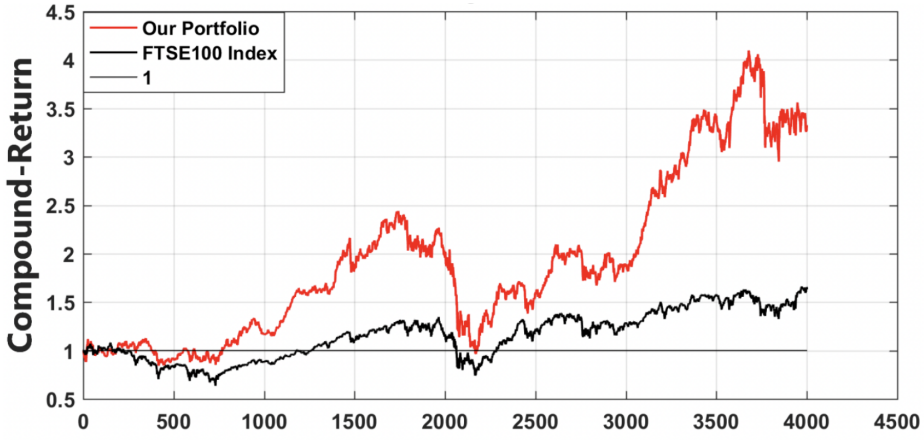


Figure: Long-term (Jan 2000 to Jun 2016)

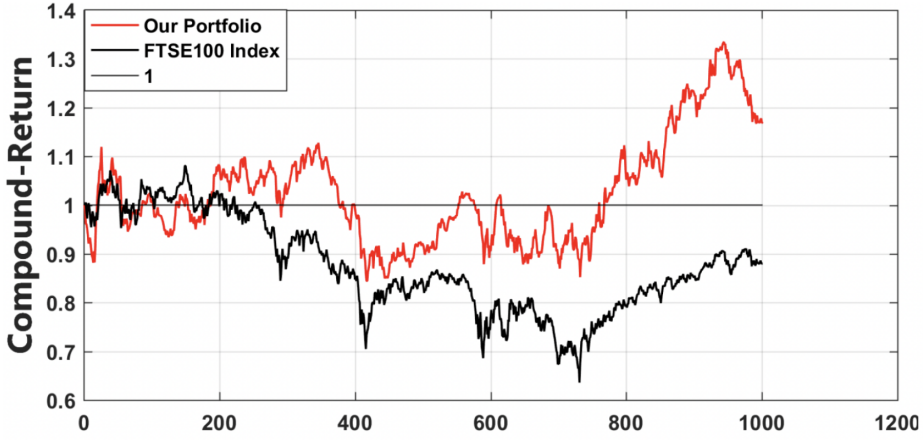


Figure: Down-Up (Jan 2000 to Jun 2004)

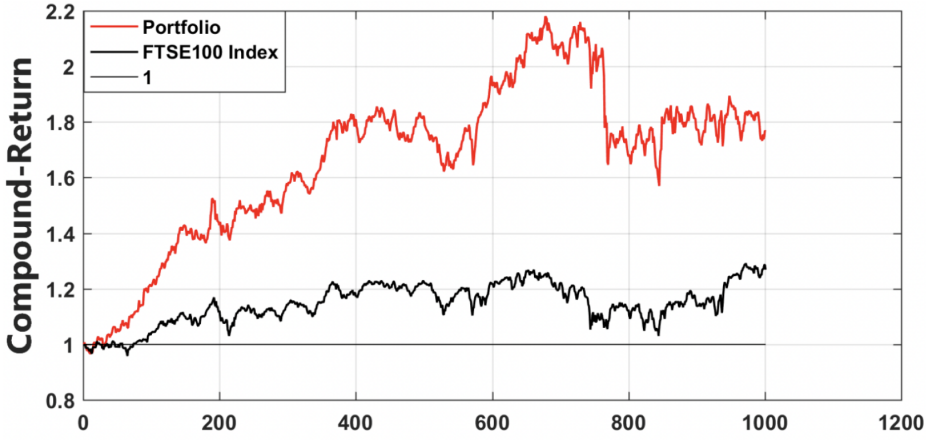


Figure: Stable (Oct 2012 to Nov 2016)

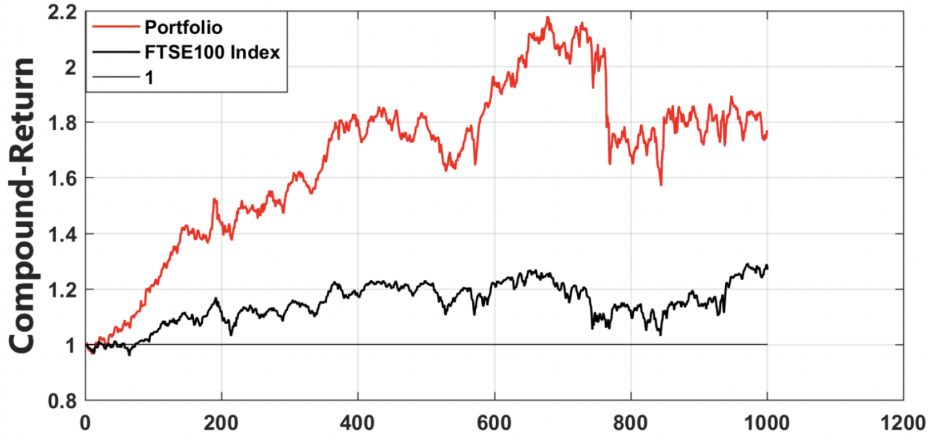


Figure: Stable (Oct 2012 to Nov 2016)

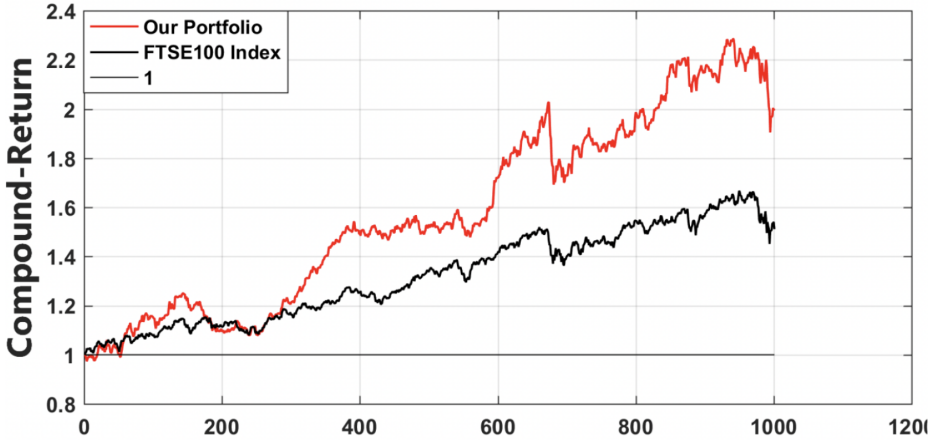


Figure: Up-Up (Jul 2003 to Jul 2007)

Measures used to test the strategy:

Measures used to test the strategy:

Sharpe Ratio: Risk-adjusted measure of return

$$S_p = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Measures used to test the strategy:

Sharpe Ratio: Risk-adjusted measure of return

$$S_p = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Maximum Drawdown (MDD): Given $t \geq 0$ and $X(0) = 0$, for a process $X(t)$,

$$\mathbf{MDD}(T) = \max_{\tau \in (0,T)} \left[\max_{t \in (0,\tau)} X(t) - X(\tau) \right]$$

Measures used to test the strategy:

Sharpe Ratio: Risk-adjusted measure of return

$$S_p = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Maximum Drawdown (MDD): Given $t \geq 0$ and $X(0) = 0$, for a process $X(t)$,

$$\mathbf{MDD}(T) = \max_{\tau \in (0,T)} \left[\max_{t \in (0,\tau)} X(t) - X(\tau) \right]$$

Total/Daily... Returns: Mean return over a period normalised by the initial value

$$r_p = \frac{V_f - V_i}{V_i}$$

V_f, V_i are adjusted based on the period considered.

Measures used to test the strategy:

Sharpe Ratio: Risk-adjusted measure of return

$$S_p = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Maximum Drawdown (MDD): Given $t \geq 0$ and $X(0) = 0$, for a process $X(t)$,

$$\mathbf{MDD}(T) = \max_{\tau \in (0,T)} \left[\max_{t \in (0,\tau)} X(t) - X(\tau) \right]$$

Total/Daily... Returns: Mean return over a period normalised by the initial value

$$r_p = \frac{V_f - V_i}{V_i}$$

V_f, V_i are adjusted based on the period considered.

Win Years: Number of years the strategy outperforms the overall index.

First, comparing the Deep Feature + Modularity (D-M) model to Raw Time Series + Modularity (R-M) and Deep Feature + k-means (D-K):

Metric	R-M	D-M (Ours)	D-K
Total Return (↑)	208.8%	**283.5%**	272.6%
Daily Sharpe (↑)	0.44	**0.50**	0.49
Max Drawdown (↑)	**-55.6%**	-60.5%	-59.0%
Daily Mean Return (↑)	9.7%	**11.1%**	10.9%
Monthly Mean Return (↑)	8.7%	**10.0%**	9.9%
Yearly Mean Return (↑)	9.6%	10.0%	**11.19%**
Winning Years (↑)	64.71%	**69.52%**	66.31%

Table: Comparison of Performance of Different Methods

First, comparing the Deep Feature + Modularity (D-M) model to Raw Time Series + Modularity (R-M) and Deep Feature + k-means (D-K):

Metric	R-M	D-M (Ours)	D-K
Total Return (↑)	208.8%	**283.5%**	272.6%
Daily Sharpe (↑)	0.44	**0.50**	0.49
Max Drawdown (↑)	**-55.6%**	-60.5%	-59.0%
Daily Mean Return (↑)	9.7%	**11.1%**	10.9%
Monthly Mean Return (↑)	8.7%	**10.0%**	9.9%
Yearly Mean Return (↑)	9.6%	10.0%	**11.19%**
Winning Years (↑)	64.71%	**69.52%**	66.31%

Table: Comparison of Performance of Different Methods

The superior performance of D-M over other methods in various key measures shows that the deeply learned feature is capturing valuable signals beyond the raw time series and that the modularity clustering method is applicable in financial settings.

Note: k-means cannot be used in practice, because of initial seed, it's not replicable.

Now, comparing the strategy with 2 big funds (CCA, VXX) and 3 best performing funds on FTSE100 (IEO, PXE, PXI) as per Yahoo Finance:

Metric	CCA	VXX	IEO	PXE	PXI	Ours
Total Return	117.0%	-99.9%	89.9%	101.6%	152.2%	**215.4%**
Daily Sharpe	0.7	-1.1	0.4	0.4	0.6	**0.8**
Max Drawdown	**-22.2%**	-99.9%	-56.8%	-57.6%	-59.3%	-30.9%
Daily Mean Return	10.9%	-67.7%	12.7%	13.4%	15.9%	**16.7%**
Monthly Mean Return	10.7%	-66.4%	11.5%	12.4%	14.9%	**16.6%**
Yearly Mean Return	6.5%	-44.6%	5.0%	8.2%	9.4%	**11.8%**
Winning Years	62.5%	0.0%	62.5%	50.0%	**75.0%**	62.5%

Table: Comparison of Performance of Strategy Against Various Funds

Now, comparing the strategy with 2 big funds (CCA, VXX) and 3 best performing funds on FTSE100 (IEO, PXE, PXI) as per Yahoo Finance:

Metric	CCA	VXX	IEO	PXE	PXI	Ours
Total Return	117.0%	-99.9%	89.9%	101.6%	152.2%	**215.4%**
Daily Sharpe	0.7	-1.1	0.4	0.4	0.6	**0.8**
Max Drawdown	**-22.2%**	-99.9%	-56.8%	-57.6%	-59.3%	-30.9%
Daily Mean Return	10.9%	-67.7%	12.7%	13.4%	15.9%	**16.7%**
Monthly Mean Return	10.7%	-66.4%	11.5%	12.4%	14.9%	**16.6%**
Yearly Mean Return	6.5%	-44.6%	5.0%	8.2%	9.4%	**11.8%**
Winning Years	62.5%	0.0%	62.5%	50.0%	**75.0%**	62.5%

Table: Comparison of Performance of Strategy Against Various Funds

In conclusion, they assert:

> *Experimental results show: (a) our learned stock feature captures semantic information and (b) our portfolio outperforms the FTSE 100 index and many well-known funds in terms of total return.*

IMPLEMENTATION

Market: S&P500 (NYSE)

Market: S&P500 (NYSE)
Time Frame: Jan 2000 - Dec 2024

Market: S&P500 (NYSE)
Time Frame: Jan 2000 - Dec 2024
Clustering Type: Networkx library default (modularity-like)

Market: S&P500 (NYSE)
Time Frame: Jan 2000 - Dec 2024
Clustering Type: Networkx library default (modularity-like)
Number of stocks: 5

Market: S&P500 (NYSE)
Time Frame: Jan 2000 - Dec 2024
Clustering Type: Networkx library default (modularity-like)
Number of stocks: 5
Asset Allocation: $1/5$ to each

Market: S&P500 (NYSE)
Time Frame: Jan 2000 - Dec 2024
Clustering Type: Networkx library default (modularity-like)
Number of stocks: 5
Asset Allocation: $1/5$ to each

Returns: Plotting the excess cumulative return (%) over the S&P500 of the portfolio:

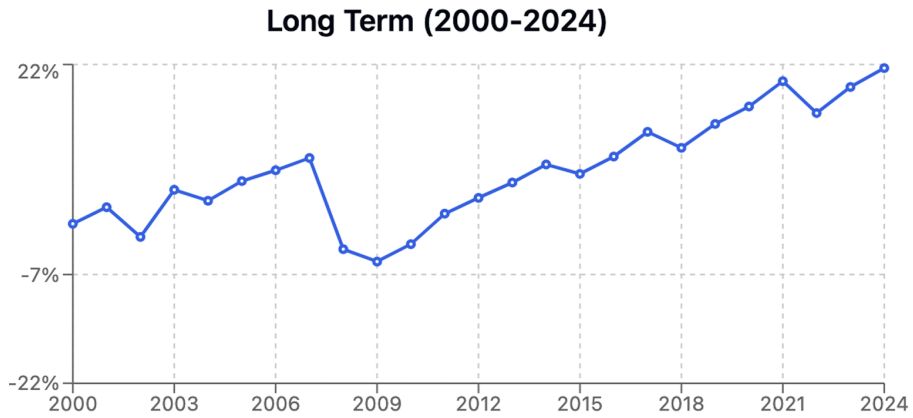
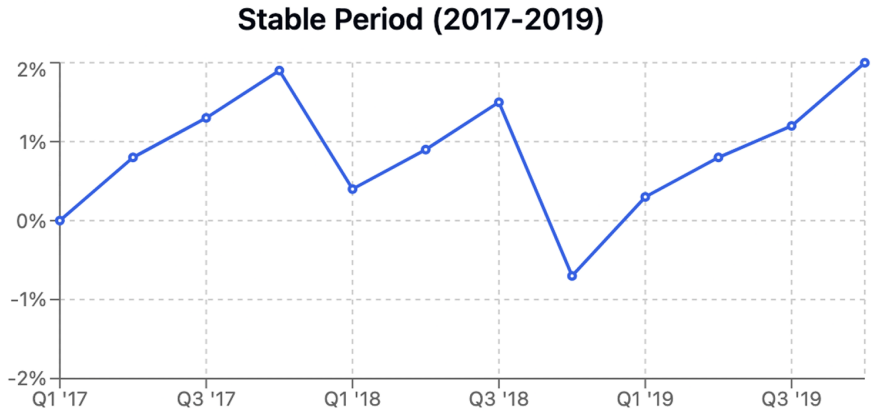
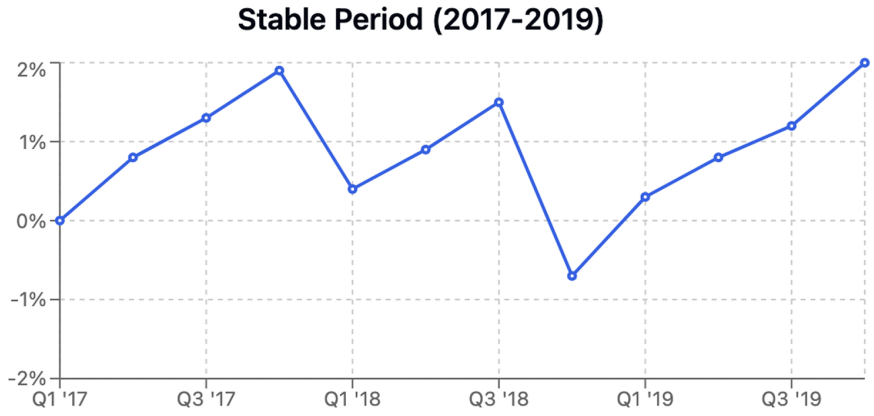
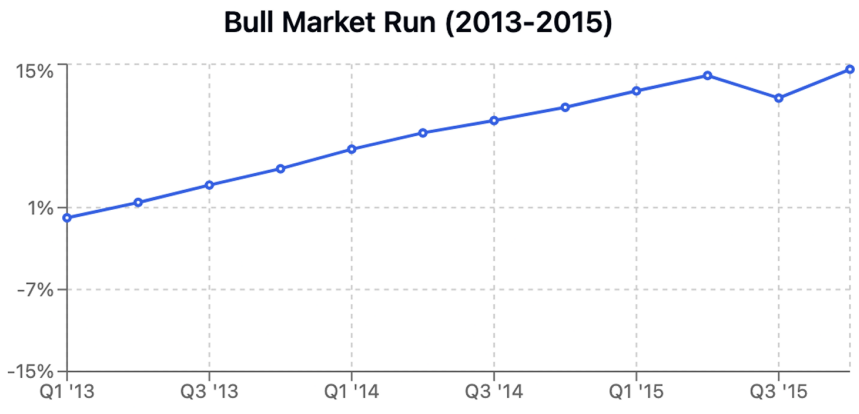
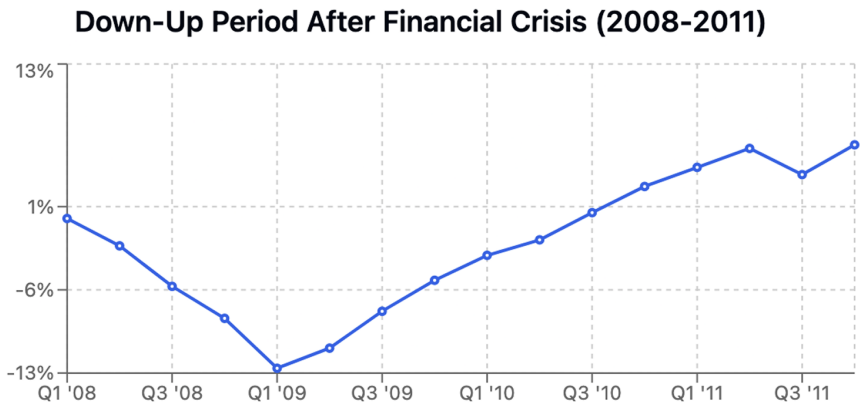


Figure: For context, the S&P500 was at $\approx 480\%$.

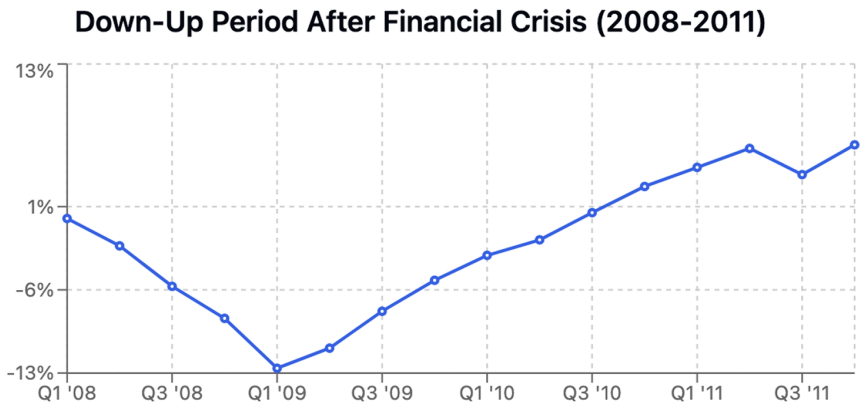

Stable Period (2017-2019)
2%
1%
0%
-1%
-2%
Q1 '17
Q3 '17
Q1 '18
Q3 '18
Q1 '19
Q3 '19


Stable Period (2017-2019)
2%
1%
0%
-1%
-2%
Q1 '17
Q3 '17
Q1 '18
Q3 '18
Q1 '19
Q3 '19


Bull Market Run (2013-2015)
15%
1%
-7%
-15%
Q1 '13
Q3 '13
Q1 '14
Q3 '14
Q1 '15
Q3 '15



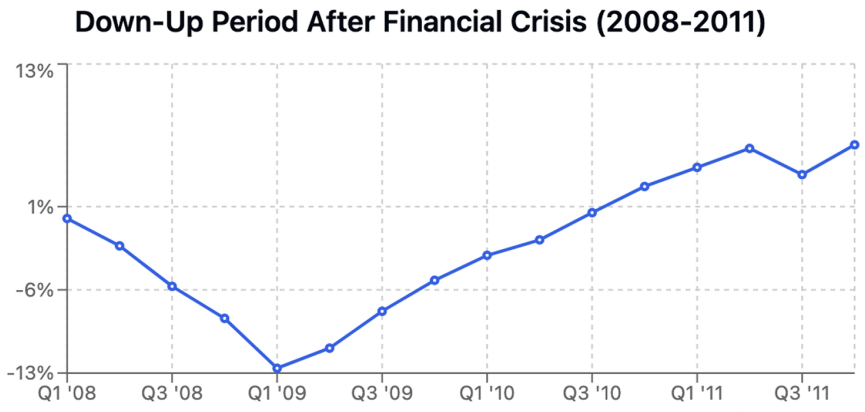
In this period, the S&P 500 had a Sharpe of 0.61, whereas the investment portfolio had a Sharpe of **0.74**. Now, a few difficulties:



In this period, the S&P 500 had a Sharpe of 0.61, whereas the investment portfolio had a Sharpe of **0.74**. Now, a few difficulties:

Implementation Challenges

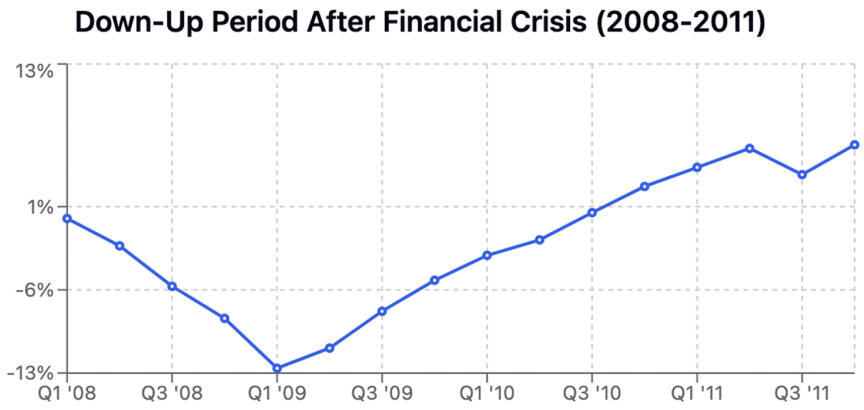
- Accounting for the frequent rebalancing, difficult to implement.



In this period, the S&P 500 had a Sharpe of 0.61, whereas the investment portfolio had a Sharpe of **0.74**. Now, a few difficulties:

Implementation Challenges

- Accounting for the frequent rebalancing, difficult to implement.
- Clustering method extremely expensive (maxed out Colab RAM)



In this period, the S&P 500 had a Sharpe of 0.61, whereas the investment portfolio had a Sharpe of **0.74**. Now, a few difficulties:

Implementation Challenges

- Accounting for the frequent rebalancing, difficult to implement.
- Clustering method extremely expensive (maxed out Colab RAM)
- Using CAE (and decoder) for 224×224 charts, difficult to debug.

CRITIQUES & FUTURE WORK

Lacking Rigorous Basis

- Why modularity-based clustering?
- Why VGG16 CAE instead of Artificial NN-based AutoEncoder
- Why pick the best stocks by recent Sharpe within a cluster?

Lacking Rigorous Basis

- Why modularity-based clustering?
- Why VGG16 CAE instead of Artificial NN-based AutoEncoder
- Why pick the best stocks by recent Sharpe within a cluster?

Not Realistic

- Piling up transaction costs due to frequent rebalancing
- Endure the computational price only to hold a single portfolio for 10 days
- Does not provide an allocation mechanism among chosen stocks

Lacking Rigorous Basis

- Why modularity-based clustering?
- Why VGG16 CAE instead of Artificial NN-based AutoEncoder
- Why pick the best stocks by recent Sharpe within a cluster?

Not Realistic

- Piling up transaction costs due to frequent rebalancing
- Endure the computational price only to hold a single portfolio for 10 days
- Does not provide an allocation mechanism among chosen stocks

Future Work

- Vary "free-parameters" like rebalancing duration, number of stocks in portfolio, encoder used, clustering method, duration of a candlestick chart, etc.
- Find a way to combine with other more recent representations that already leverage LLMs and conventional methods.